

Lab: Socket Programming

In this lab, we will write Python programs to create a client and server and explore how they communicate with each other using the concept of socket programming. Recalling what we studied in the previous Lab, and the next lab, we will see how a server responds to client requests. Therefore, we'll develop our own programs using Python's socket library. Additionally, we will be introduced to the concept of multi-threading, enabling our server to handle requests from multiple clients simultaneously and respond appropriately.

WHAT IS SOCKET PROGRAMMING?

A socket is a way for programs to communicate with each other over a network. It's like a phone line that lets computers send and receive data.

Socket programming involves writing programs that run on different computers and communicate over a network. This can be either connection-oriented (like making a phone call) or connectionless (like sending a letter). The basic way to do this is to import the socket library in Python, then create a socket object and then bind that object with a socket address. A socket address is a combination of an IP address and a port number. It uniquely identifies a network connection on a specific device. For example, a device with an IPv4 address of 192.168.10.1 and port number 5050 will have a socket address of 192.168.10.1:5050.

What is Socket Programming?

- The term *socket programming* refers to writing programs that execute across multiple computers in which the devices are all connected to each other using a network.
- Socket programming can be connection-oriented or connection-less.
- Python uses its `socket` library to implement socket programming.
- Transmission Control Protocol (TCP) is used as a default protocol.

PYTHON'S SOCKET LIBRARY

Python provides a socket library to make socket programming easier. Here are some important methods:

- **`socket()`**: Creates a new socket object.
- **`bind((hostname, port))`**: Binds the server to a specific IP address and port number.
- **`listen(backlog)`**: Prepares the server to accept connections. The backlog defines how many connections can wait in line.
- **`accept()`**: Accepts a new connection. Returns a new socket object and the address of the client.
- **`connect((hostname, port))`**: Connects the client to the server at the specified address.
- **`send(message)`**: Sends data to the connected socket.
- **`recv(buffer_size)`**: Receives data from the connected socket.
- **`close()`**: Closes the socket connection.

Client-side Socket Programming

- We use the socket object to make connection to the server, send data to and receive data from the server.
- Steps involved in a typical communication with the server –
 1. First, `socket` library is imported and `socket()` object is created.
 2. The client initiates connection to a server specified by hostname/IP address and port number using `connect((hostname/ip_address, port_number))` method.
 3. Send data to the server using `send(message)` method.
 4. Read data from the server using `recv(buffer_size)` method.
 5. Close the connection using `close()` method.

Client-side Socket Programming

- Method Description:
 - `socket.socket(socket.AF_INET, socket.SOCK_STREAM) :`
Here, `AF_INET` refers to the address-family ipv4.
The `SOCK_STREAM` means connection-oriented TCP protocol.
 - `connect((Hostname/Ip Address, Port Number)) :`
Here, the tuple takes hostname or ip address of the server and the port number.
 - `send(message) :`
Here, the given message needs to be encoded using an encoding format. E.g., `message.encode('utf-8')`.
 - `recv(buffer_size) :`
Here, the return value is a bytes object representing the data received. It needs to be decoded using appropriate format. E.g., `recv(2048).decode('utf-8')`.

Server-side Socket Programming

- We use the socket object to make connection with the client, send data to and receive data from the client.
- Steps involved in a typical communication with the client–
 1. First, `socket` library is imported and `socket()` object is created.
 2. Bind the server socket with hostname/IP address and port number using `bind((hostname/ip_address, port_number))` method.
 3. Listen for a connection from the client using `listen()` method and accept it using `accept()` method. This results in a client socket being created for the connection.
 4. Use the client socket to send data to the client and receive data from the client.
 5. Close the connection using `close()` method.

Server-side Socket Programming

- Method Description:

- **`listen(backlog)` :**

- Enable a server to accept connections. The backlog specifies the number of unaccepted connections that the system will allow before refusing new connections. If not specified, a default reasonable value is chosen.

- **`accept()` :**

- Accept a connection. The return value is a pair `(conn, address)` where `conn` is a new socket object usable to send and receive data on the connection, and `address` is the address bound to the socket on the other end of the connection.

Socket Programming

■ Other Methods:

- `socket.gethostname()` :

This returns a string containing the *hostname* of the machine where the Python interpreter is currently executing.

- `socket.gethostbyname()` :

Translate a host name to IPv4 address format. The IPv4 address is returned as a string, such as '100.50.200.5'. If the host name is an IPv4 address itself it is returned unchanged.

How it works

localhost:8085

Client

```
host= socket.gethostbyname(socket.gethostname())  
port = 8085  
addr = (host, port)
```

localhost:6666

Server

```
host= socket.gethostbyname(socket.gethostname())  
port = 6666  
addr = (host, port)
```

IP address can be found from host part.

How it works

localhost:8085

Client

```
client = socket.socket(socket.AF_INET,  
socket.SOCK_STREAM)
```

localhost:6666

Server

```
server = socket.socket(socket.AF_INET,  
socket.SOCK_STREAM)
```

```
server.bind(addr)
```

Socket object is created on both the client and server side. The address which was created lastly is then bound to the socket using the `server.bind` method.

How it works

localhost:8085

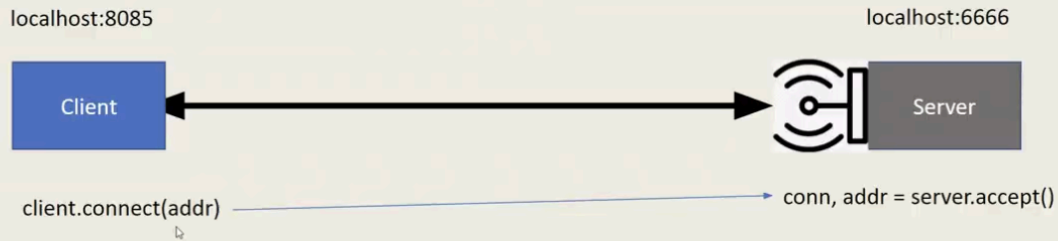
Client

localhost:6666



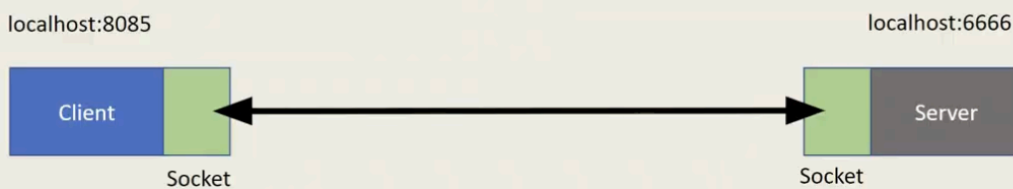
```
server.accept()
```

How it works



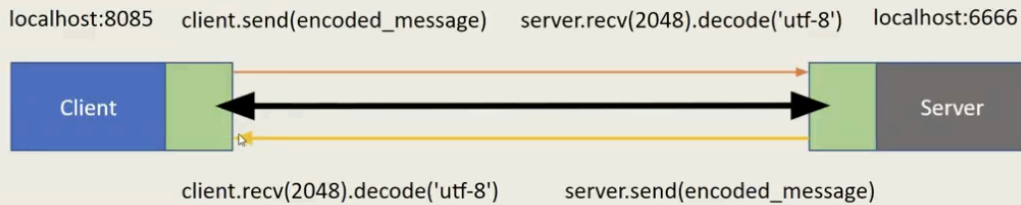
Conn refers to the object which will be used for data transmission and addr is the address of the client which is trying to connect to the server.

How it works



2 sockets are established on both sides of the connection.

How it works



MULTI-THREADING CONCEPT

In our initial tasks, we will see the communication between one server and one client. But in real life, multiple clients request the same server at the same time. It is not feasible and efficient to build individual servers for each client. So, what can we do in this case? This is when we apply the concept of multi-threading. Multi-threading allows a server to handle multiple client requests at the same time. This means the server can talk to many clients without making them wait for each other.

We can implement this by importing another Python library named 'threading'. It allows the server to handle multiple client connections at the same time by creating a separate thread for each connection. A new thread is created specifically to handle the communication with the connected client. This thread can send and receive messages independently for that specific client. Each thread runs independently, enabling the server to manage multiple clients concurrently without waiting for one connection to finish before starting another. Furthermore, if one client's connection is slow or blocked, it doesn't affect the handling of other clients.

OUTCOME

By the end of this lab, you should be able to write Python programs that use sockets to communicate over a network and handle multiple clients at the same time.

Class Work

1. Create a basic client-server program where the client informs the server of the client's IP and device name whenever a client is connected and the server prints that.

Home Work

2. Create a basic client-server program where the server receives a message from the client and counts how many vowels are there in the message. If there are no vowels in the message then the server sends "Not enough vowels" message to the client, if there are at most two vowels then the server sends "Enough vowels I guess" and if there are more than two vowels then the server sends "Too many vowels" to the client.

3. Create a basic **multi-threaded** client server program where the server receives a message from the client and counts how many vowels are there in the message. If there are no vowels in the message then the server sends "Not enough vowels" message to the client, if there are at most two vowels then the server sends "Enough vowels I guess" and if there are more than two vowels then the server sends "Too many vowels" to the client.

Only difference here is that the server will be able to respond to multiple clients simultaneously.



United International University
Dept. of Computer Science & Engineering
Class Test - 2

Course Code: CSE 3712

Course Title: CN Lab

Time: 30 min

Marks: 20

1. Create a basic client-server program where the server receives a message from the client and counts how many vowels are there in the message. If there are no vowels in the message then the server sends “Not enough vowels” message to the client, if there are at most two vowels then the server sends “Enough vowels I guess” and if there are more than two vowels then the server sends “Too many vowels” to the client.

2. Create a basic **multi-threaded** client server program where the server receives a message from the client and counts how many vowels are there in the message. If there are no vowels in the message then the server sends “Not enough vowels” message to the client, if there are at most two vowels then the server sends “Enough vowels I guess” and if there are more than two vowels then the server sends “Too many vowels” to the client.

Only difference here is that the server will be able to respond to multiple clients simultaneously.